

Pauli Correlation Encoding for Budget-Constrained Optimization

Jacobo Padín-Martínez,¹ Vicente P. Soloviev,² Alejandro Borrallo-Rentero,¹
 Antón Rodríguez-Otero,¹ Raquel Alfonso-Rodríguez,¹ and Michal Krompiec²

¹*International Quantum Center - FSAS Technologies (Fujitsu)*

²*Fujitsu Research of Europe Ltd.*

Quantum optimization has gained increasing attention as advances in quantum hardware enable the exploration of problem instances approaching real-world scale. Among existing approaches, variational quantum algorithms and quantum annealing dominate current research; however, both typically rely on one-hot encodings that severely limit scalability. Pauli Correlation Encoding (PCE) was recently introduced as an alternative paradigm that reduces qubit requirements by embedding problem variables into Pauli correlations. Despite its promise, PCE has not yet been studied in the context of constrained optimization. In this work, we extend the PCE framework to constrained combinatorial optimization problems and evaluate its performance across multiple problem sizes. Our results show that the standard PCE formulation struggles to reliably enforce constraints, which motivates the introduction of the Iterative- α PCE. This iterative strategy significantly improves solution quality, achieving consistent constraint satisfaction while yielding better cut sizes across a wide range of instances. These findings highlight both the limitations of current PCE formulations for constrained problems and the effectiveness of iterative strategies for advancing quantum optimization in the NISQ era.

I. INTRODUCTION

Quantum optimization has emerged as a prominent area of research within quantum computing over recent decades [1]. The steady increase in the number of qubits available in quantum processing units (QPUs) has made it possible to address and solve problem instances that more closely resemble real-world use cases, bridging the gap between classical computational approaches and industrial applications.

Among the most established approaches, variational quantum algorithms (VQAs) leverage tunable quantum circuits optimized through classical feedback loops, making them particularly suitable for near-term noisy devices [2]. An example of such algorithms is the Quantum Approximate Optimization Algorithm (QAOA) [3], designed to approximate solutions to NP-hard graph-based problems. Other examples of VQAs include the Variational Quantum Eigensolver (VQE) [4], traditionally used for quantum chemistry problems. Quantum annealing (QA) [5] represents another influential paradigm, relying on adiabatic evolution to guide a quantum system toward the ground state of a problem Hamiltonian; this method has been widely explored in both academic research and industry [6].

Most of the approaches discussed above share a common characteristic: the mapping from classical optimization variables to qubits is typically performed through a one-hot encoding, whereby each variable is assigned to a qubit in a one-to-one manner. This strategy becomes a significant limitation when scaling to realistic scenarios involving a large number of variables. Recently, the Pauli Correlation Encoding (PCE) [7] algorithm was proposed to overcome this limitation by allowing the number of qubits in the system to be substantially smaller than the number of optimization variables. This is achieved by encoding variable information into expectation values of

Pauli observables rather than directly into the quantum state of individual qubits.

Given its recent introduction, the literature on PCE remains limited. Nevertheless, the approach has already been applied to realistic financial scenarios, such as portfolio optimization [8], and its adoption is expected to grow, as major platforms are dedicating specific tutorials and resources to the method [9]. Despite this growing interest, systematic studies of PCE remain scarce, making further theoretical and empirical insights particularly valuable.

To the best of our knowledge, constrained optimization problems have not yet been analyzed within the PCE framework. In this work, we extend PCE to a QUBO with constraint, focusing specifically on the budget-constrained mincut problem. We perform a detailed analysis of how the algorithm performs and adapts in the presence of constraints and show that, in its standard formulation, PCE exhibits limited accuracy when targeting constrained optimization problems. To address this limitation, we propose an iterative variant that significantly improves solution accuracy and constraint satisfaction, consistently outperforming the standard PCE.

The outline of the paper is as follows: Section II describes the PCE approach in detail; Section III presents the specific problem used in the experimentation; Sections IV provides the results and analysis of the PCE baseline; Section V introduces the novel iterative PCE method and additional benchmarking results; finally, Section VI concludes the paper with further considerations and directions for future research.

II. PAULI CORRELATION ENCODING

The Pauli Correlation Encoding (PCE) is a quantum algorithm designed to solve combinatorial prob-

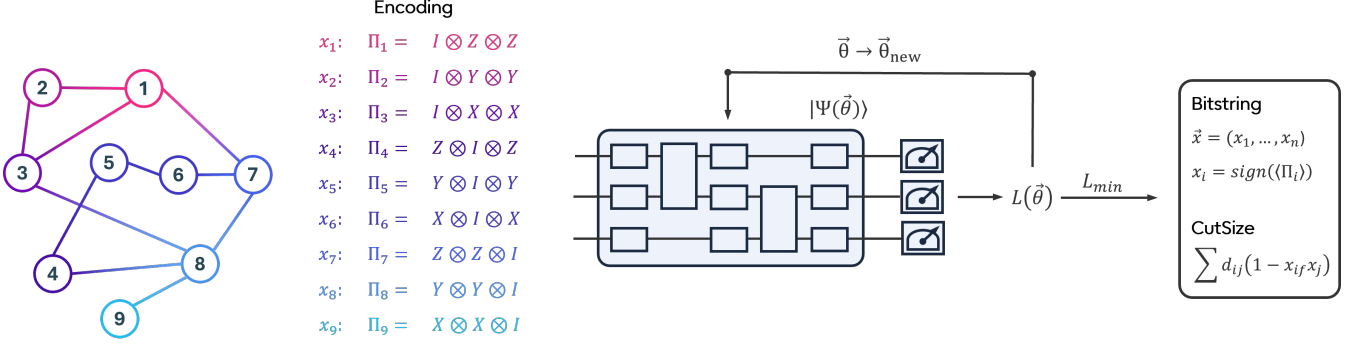


FIG. 1: Pauli Correlation Encoding (PCE) optimization scheme.

lems (originally introduced for QUBO problems) on gate-based quantum computers. Unlike approaches such as QAOA, which encode binary variables in the computational basis states of qubits, PCE encodes the variables in the signs of the expectation values of a predefined set of Pauli string operators. These operators are constructed as tensor products of Pauli matrices, with each string containing only a single type of Pauli matrix:

$$\Pi_i \in \{X^{\otimes k} \otimes \mathbb{I}^{\otimes m-k}, Y^{\otimes k} \otimes \mathbb{I}^{\otimes m-k}, Z^{\otimes k} \otimes \mathbb{I}^{\otimes m-k}\} \quad (1)$$

or any of their permutations, where k is the order (number of Pauli matrices per string), and m is the number of qubits. For example, in a three-qubit system with $k = 2$, the possible operators include $\{IXX, IYY, IZZ, XIX, YIY, ZIZ, XXI, YYI, ZZI\}$, effectively encoding 9 variables using only 3 qubits.

In the original PCE paper [7], the authors applied this approach to solve Max-Cut problems, formulated as the minimization of the following QUBO:

$$\text{MaxCut} = \sum_{i>j} \sum_{j=0}^{n-1} d_{ij} (1 - x_i x_j), \quad (2)$$

where d_{ij} denotes the weight of the edge that connects nodes i and j , and $x_i \in \{0, 1\}$ represents the binary variable associated with node i , indicating which side of the cut the node is assigned. The objective function corresponds to the cut value, defined as the sum of the weights of the edges connecting nodes belonging to different partitions. For instance, if $x_i = 0$ and $x_j = 1$, nodes i and j are placed on opposite sides of the cut and the corresponding edge contributes to the objective function. In the PCE algorithm, binary variables are substituted by the sign function, i.e. $x_i = \text{sign}(\langle \Pi_i \rangle)$:

$$\text{MaxCut} = \sum_{i>j} \sum_{j=0}^{n-1} d_{ij} [1 - \text{sign}(\langle \Pi_i \rangle) \text{sign}(\langle \Pi_j \rangle)] \quad (3)$$

However, in practice, the sign function is replaced by the hyperbolic tangent, which provides a smooth approximation of the sign function, better suited for classical optimizers [10]. Thus, the loss function is finally formulated

as follows:

$$L = \sum_{i>j} \sum_{j=0}^{n-1} d_{ij} [1 - \tanh(\alpha \langle \Pi_i \rangle) \tanh(\alpha \langle \Pi_j \rangle)] + L^{\text{reg}} \quad (4)$$

where L^{reg} is a regularization term that will be discussed later. The optimization workflow, detailed in Figure 1, is as follows:

1. Prepare a parametrized quantum state $|\Psi(\vec{\theta})\rangle$.
2. Compute the expectation values $\langle \Pi_i \rangle$ to evaluate the loss function $L(\vec{\theta})$.
3. Use a classical optimizer to update the parameters for the next iteration.
4. Repeat the procedure until the loss is minimized.
5. Once convergence is reached, construct the bitstring solution $x = (x_1, x_2, \dots, x_n)$ with $x_i = \text{sign}(\langle \Pi_i \rangle)$.
6. Compute the cut size value using the bitstring solution in the original QUBO expression Eq. (5).

III. BUDGET-CONSTRAINED MINCUT PROBLEMS

Many real-world optimization problems involve additional constraints that go beyond the standard unconstrained MinCut or MaxCut formulations. In particular, budget or balance constraints naturally arise in applications where the two partitions induced by the cut must satisfy predefined size, capacity, or cost requirements [11–14]. Examples include resource allocation [15], and clustering problems [16]. In this study we will focus on the mincut problem with a budget constraint term.

First, a MinCut problem can be formulated as minimizing the following QUBO problem:

$$\text{MinCut} = \sum_{i=1}^{n-1} \sum_{j=i+1}^n d_{ij} (x_i - x_j)^2 \quad (5)$$

A budget-constrained MinCut is formulated by introducing a constraint term as follows:

$$\text{MinCut} = \sum_{i=1}^{n-1} \sum_{j=i+1}^n d_{ij} (x_i - x_j)^2 + \beta \left(\sum_{i=1}^n x_i - c \right)^2 \quad (6)$$

where c denote the number of nodes assigned to one of the subgroups. For example, the divisions 1 : 7, 2 : 6, 3 : 5 and 4 : 4 can be obtained by setting $c = 1, 2, 3, 4$ (or equivalently 7, 6, 5, 4). In general, for a system with n nodes, $c \in [1, n/2]$, since the values $c > N/2$ reproduce the same divisions, i.e., a division $c : N - c$ is equivalent to $N - c : c$.

To solve the budget-constrained MinCut QUBO using the PCE, first it has to be expressed in terms of binary variables $z_i = \{-1, 1\}$. Let $x_i = \{0, 1\}$, the change of variables is defined as:

$$x_i = \frac{z_i + 1}{2} \quad (7)$$

the cut value term can be rewritten as:

$$\begin{aligned} \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{1}{4} d_{ij} (z_i - z_j)^2 &= \\ &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{1}{4} d_{ij} (-2z_i z_j + z_i^2 + z_j^2) \end{aligned} \quad (8)$$

since $z_i^2 = 1$, this simplifies to:

$$\sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{1}{2} d_{ij} (1 - z_i z_j) \quad (9)$$

The constraint term, applying the same change of variables yields:

$$\begin{aligned} \beta \left(\sum_i^n x_i - c \right)^2 &= \beta \left[\frac{1}{2} \left(\sum_i^n z_i + 1 \right) - c \right]^2 = \\ &= \beta \left[\frac{1}{2} \left(\sum_i^n z_i + 1 - 2c \right) \right]^2 \end{aligned} \quad (10)$$

ignoring the constant factor $1/2$, which does not affect the optimization process:

$$\beta \left[\left(\sum_i^n z_i \right) + n - 2c \right]^2 \quad (11)$$

the last step is to do the substitution $z_i = \tanh(\langle \alpha \Pi_i \rangle)$, obtaining the final expression:

$$\begin{aligned} L &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{1}{2} d_{ij} [1 - \tanh(\langle \alpha \Pi_i \rangle) \tanh(\langle \alpha \Pi_j \rangle)] + \\ &+ \beta \left[\sum_i^n \tanh(\langle \alpha \Pi_i \rangle) - (n - 2c) \right]^2 \end{aligned} \quad (12)$$

The regularization term is not included in this expression.

The fundamental challenge in applying Pauli Correlation Encoding (PCE) to constrained cut problems stems from its inherently non-binary nature. Algorithms such as the Quantum Approximate Optimization Algorithm (QAOA) are intrinsically tailored to binary optimization, as their solutions are obtained through qubit measurements that naturally yield binary outcomes in the computational basis, resulting in a bitstring representation of the solution. In an ideal setting, PCE would exhibit analogous behavior with the sign function, however, in practice, the sign function is relaxed to $\tanh(\cdot)$, allowing the encoded variables to take continuous values in the interval $[-1, 1]$ rather than being restricted to the discrete set $\{-1, 1\}$.

This relaxation gives rise to two critical consequences. First, the cut-value term in the loss function is minimized over a continuous domain. As a result, configurations of the variables with decimal values can yield lower loss values than any valid binary configuration, potentially leading to incorrect cut assignments once the solution is discretized. Second, constraint terms may fail to be satisfied, giving solutions which effectly minimizes the quantity in the real domain, but however, it may does not satisfy the proper condition. In fact, for a fixed value of c , the constraint defined in Eq. 12 is fulfilled when the sum of the $\tanh(\langle \alpha \Pi_i \rangle)$ variables equals $n - 2c$. In the binary case, where $\tanh(\langle \alpha \Pi_i \rangle) \in \{-1, 1\}$, this condition guarantees that exactly c nodes are assigned to one side of the cut. In the continuous regime, however, this equality is generally not enforced: instead, the optimizer seeks a real-valued configuration that minimizes the constraint penalty, which does not necessarily correspond to selecting precisely c nodes.

In this paper we will analyze and address this problematic, by using the PCE on the budget-constrained MinCut problems, where the cut must satisfy a global constraint on the number of nodes assigned to each partition.

IV. PCE ANALYSIS

In this section, we analyze the PCE algorithm in the context of the budget-constrained MinCut optimization problem described in Section III. We focus on how each component of the algorithm influences the solution, with particular emphasis on constraint satisfaction and its relationship with the final values of the variables. The graph instances used throughout this section are summarized in Table I. The ansatz employed in this work is the same as that used in the original PCE study, the Brickwork Ansatz, and only a single layer is used. For the analysis of the results, we introduce the following metrics:

1. **Constraint success ratio:** Denoted as ε_c , this value represents the efficiency with which the algorithm fulfills the constraint. Given N executions

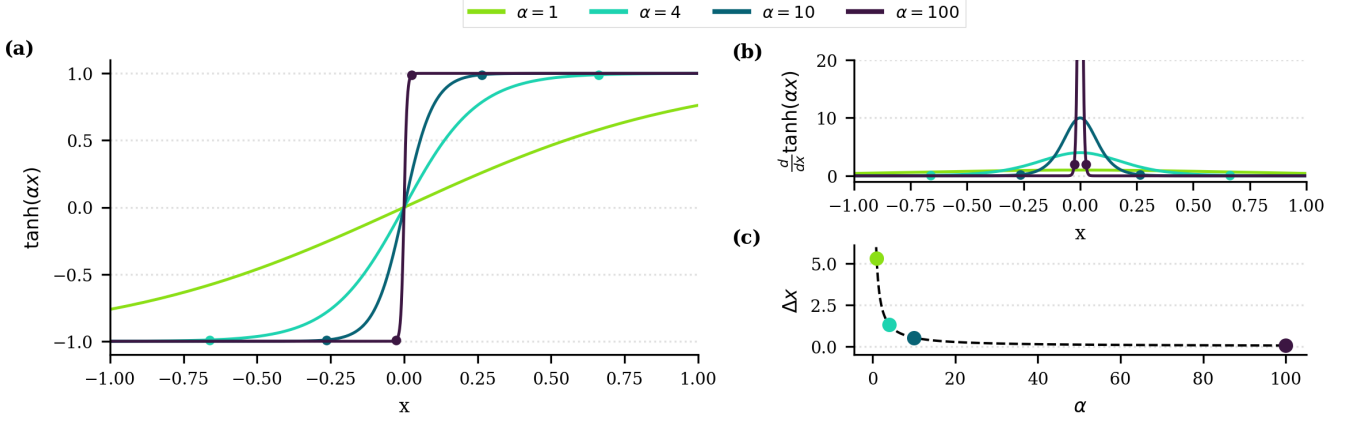


FIG. 2: (a) $\tanh(\alpha \langle \Pi \rangle)$ function. (b) $\tanh(\alpha \langle \Pi \rangle)$ derivative. In both plots a point is marked where $\tanh(\alpha \langle \Pi_i \rangle) = \pm 0.99 = x_{\pm 0.99}$. (c) Width of the plateau region $\Delta x = x_{+0.99} - x_{-0.99}$ vs α is shown. The points highlighted are for those α in the plots

of the PCE, being N_c the number of executions in which the cut satisfied the constraint, this ratio is:

$$\varepsilon_c = \frac{N_c}{N} \quad (13)$$

If $\varepsilon_c = 0$ there was not any execution which satisfied the constraint, and therefore there is not any valid solution. If $\varepsilon_c = 1$ all executions satisfied the constraint and every solution is valid.

- 2. Binarization:** The ratio of variables that effectively attain binary values after optimization. We consider that a variable $\tanh(\alpha \langle \Pi_i \rangle)$ is binarized if $|\tanh(\alpha \langle \Pi_i \rangle)| > 0.9$. Defining V as the set of binarized variables:

$$V = \{i \mid |\tanh(\alpha \langle \Pi_i \rangle)| > 0.9\}$$

the binarization is computed as:

$$\text{Binarization} = \frac{|V|}{n} \quad (14)$$

If Binarization = 0 means no variable was binarized and all remained in the real regime between $[-0.9, 0.9]$. On the contrary, if Binarization = 1 all the variables were binarized.

- 3. CutSize:** The cut size value given by the bitstring solution $z = (z_1, z_2, \dots, z_n)$:

$$\text{CutSize} = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{1}{2} d_{ij} (1 - z_i z_j) \quad (15)$$

To facilitate comparisons across different graph instances and values of the constraint parameter c , all CutSize values are reported normalized with respect to the solution obtained for the same graph and constraint using simulated annealing [17]. This

reference solution provides a high-quality baseline against which the performance of the PCE algorithm can be evaluated.

In all the results presented in this work, the CutSize is computed only for those simulations in which the constraint is satisfied. This restriction ensures that these metrics reflect the quality of valid solutions exclusively. Since both quantities are intended to evaluate the performance of the algorithm in producing meaningful cuts, it would be inconsistent to include cases where the constraint is violated, as such configurations do not represent feasible solutions to the problem.

Nodes	Edges	Density	Avg.Degree	Clustering	Connected
6	15	1.00	5.00	1.00	✓
14	91	1.00	13.00	1.00	✓
18	153	1.00	17.00	1.00	✓
20	190	1.00	19.00	1.00	✓
25	300	1.00	24.00	1.00	✓

TABLE I: Key structural properties of the analyzed graphs. The final column indicates whether each graph forms a single connected component.

A. Parameter α

The parameter α controls the smoothness of the effective sign function. Since the expectation values are bounded within the interval $[-1, 1]$, the parameter α must be sufficiently large for the hyperbolic tangent to approach its asymptotic values of -1 and 1 within this interval. For instance, using $\alpha = 1$ precludes reaching this limits, resulting in insufficient binarization (see Figure 2). We evaluated the PCE algorithm over a wide range of values $\alpha \in [4, 1000]$. Figure 3 illustrates the impact of α on

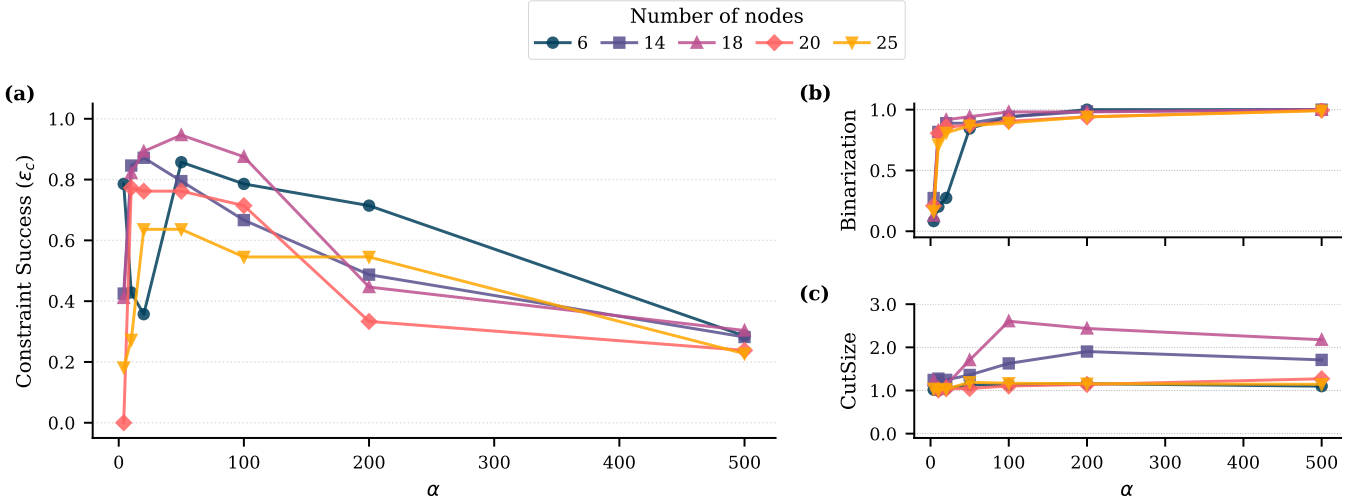


FIG. 3: (a) Constraint success ratio ϵ_c as a function of α . (b) Binarization as a function of α . (c) CutSize as a function of α . Each graph was executed 10 times for all values of the constraint parameter $c \in [2, m/2]$, where m denotes the number of nodes. Reported values correspond to averages over all runs. The classical optimizer used was SLSQP. The penalty parameter was fixed to $\beta = 1000$.

both the constraint success ratio ϵ_c and the binarization. For low values of α , both ϵ_c and the binarization remain low, increasing progressively as α grows. The maximum value of ϵ_c is achieved in the same range where the binarization approaches unity. Beyond this regime, larger values of α lead to fully binarized variables; however, the constraint success ratio is drastically lower. In addition, the CutSize is observed to increase with α , particularly for the 14-node and 18-node graph instances, indicating that the algorithm struggles to identify better minima. This behavior reflects an intrinsic limitation of approximating the sign function. Beyond a certain value of α , the optimizer fails to identify parameter configurations that further minimize the constraint term. Moreover, even in cases where the constraint is satisfied, the algorithm frequently converges to suboptimal solutions.

The optimizer employed in our experiments is the same as that used in the original PCE work, the SLSQP algorithm. This optimizer determines the optimization trajectory by exploiting gradient information of the loss function. In the PCE, the loss depends explicitly on the derivatives of the $\tanh(\cdot)$ function. As α increases, the derivative rapidly vanishes over a broad region of the variable domain, as shown in Figure 2(b).

To further investigate whether this behavior is related to the use of a gradient-dependent optimizer such as SLSQP, we repeated the same analysis using a gradient-free method, the Nelder-Mead algorithm [18]. The corresponding results are shown in Figure 4. A qualitatively similar behavior is observed when using Nelder-Mead, indicating that the observed effect is independent of the use of gradient information. Consequently, the limitation cannot be attributed to the gradients evaluating to zero. Instead, it arises from the structure of the loss landscape

itself. As α increases, the effective region of the parameter space that meaningfully influences the loss function becomes increasingly narrow. This effect is illustrated in Figure 2(c), where the width of the plateau associated with the tanh function is shown as a function of α . As α grows, this width shrinks toward zero, severely restricting the region in which parameter variations lead to appreciable changes in the loss. As a result, both gradient-based and gradient-free optimizers are unable to efficiently explore the landscape, leading to premature convergence to suboptimal solutions.

Therefore, the optimal value of the parameter α must be sufficiently large to ensure that the variables are properly binarized and that the resulting solution satisfies the constraint, while at the same time avoiding stalling of optimization in flat regions of the landscape. The original PCE paper suggests that the parameter α scales as $\alpha \sim n^{k/2}$, where n denotes the number of qubits and k the number of Pauli operators per Pauli string. In this experiments we have used $k = 2$. Table II reports the performance of the algorithm for two representative values of α : the value suggested in the original work, $\alpha \sim n^{k/2} \sim n \sim 4$, and a significantly larger value, $\alpha = 100$. In terms of the constraint success ratio ϵ_c , the smaller value $\alpha = 4$ performs substantially worse than $\alpha = 100$ as the size increases. This can be directly attributed to the insufficient binarization, much lower than 100. However, when evaluating the cut size, larger values of α do not systematically lead to improved solutions. In fact, $\alpha = 100$ yields slightly worse cut sizes in most cases, with the exception of the 18-node graph, where the resulting cut size is approximately twice that obtained for $\alpha = 4$. In summary, α constitutes a critical hyperparameter in the PCE algorithm. While strong binarization

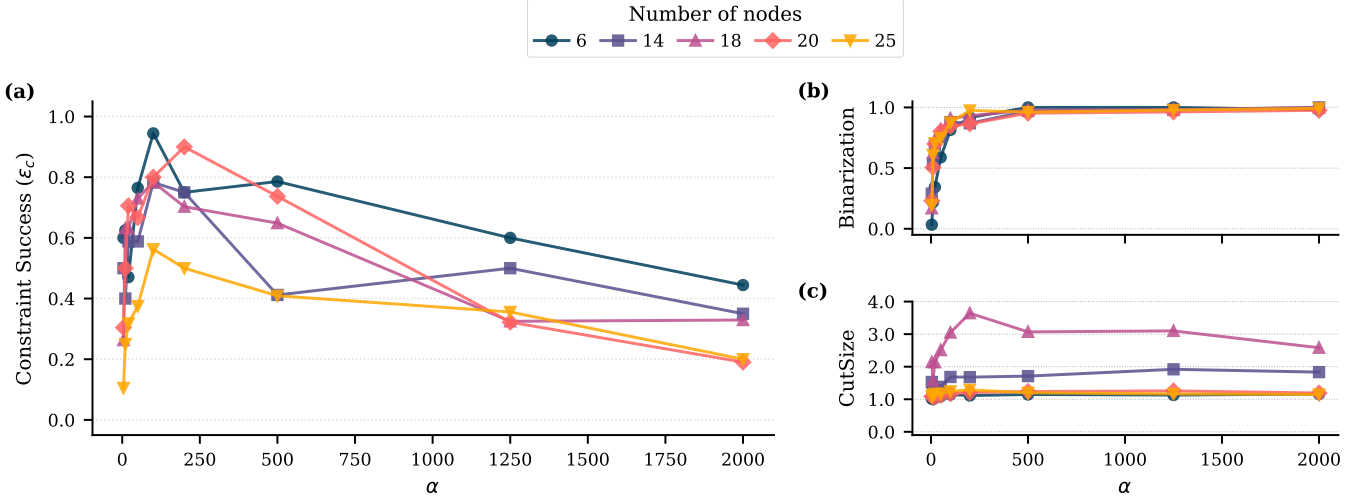


FIG. 4: (a) Constraint success ratio ϵ_c as a function of α . (b) Binarization as a function of α . (c) CutSize as a function of α . Each graph was executed 10 times for all values of the constraint parameter $c \in [2, m/2]$, where m denotes the number of nodes. Reported values correspond to averages over all runs. The classical optimizer used was Nelder-Mead. The penalty parameter was fixed to $\beta = 1000$.

seems to be a necessary condition for satisfying the constraint, it is not sufficient.

Nodes	α	ϵ_c	Binarization	CutSize
6	4	0.79	0.08	1.02
6	100	0.79	0.94	1.13
14	4	0.43	0.27	1.24
14	100	0.67	0.94	1.63
18	4	0.41	0.13	1.24
18	100	0.88	0.98	2.61
20	4	0.00	0.21	-
20	100	0.71	0.90	1.10
25	4	0.18	0.16	1.04
25	100	0.55	0.89	1.17

TABLE II: Performance comparison of PCE for two values of the hyperparameter α . Entries marked with ‘-’ indicate cases where no feasible solution satisfying the constraint was found.

B. Penalty parameter β

The parameter β controls the strength of the penalty applied when the cut violates the constraint, i.e., when the solution does not separate exactly c nodes. Selecting appropriate values for penalty parameters is a well-known challenge in combinatorial optimization and remains an open problem in both classical and quantum optimization methods [19]. We have tested the performance of the PCE for several combinations of the parameters α, β and the constraint parameter c . Figure 5 illustrates the

results. The main observations can be summarized as follows:

1. For each value of c , the overall performance of PCE is primarily governed by the choice of α .
2. For fixed values of α and c , there is no clear range of β values that consistently yields optimal performance. In practice, similar performance levels are observed for $\beta \sim 10^2$, $\beta \sim 10^3$, and $\beta \sim 10^4$.
3. At fixed α and c , the performance is highly sensitive to the specific value of β within the same order of magnitude. For example, for $\alpha = 4$ and $c = 5$, a peak in the constraint success ratio ϵ_c is observed at $\beta = 5000$, whereas a slight increase to $\beta = 7000$ causes ϵ_c to drop to zero. Several analogous cases are visible in the figure.
4. The value of β that yields the best performance is not transferable across different (α, c) configurations. This is evident from the highlighted bars in Figure 5, which indicate the highest ϵ_c for each configuration. For instance, while $\beta = 3000$ and $\beta = 70000$ provide the best results for $\alpha = 4$ and $c = 7$, these same values lead to significantly poorer performance for $c = 3$.

Therefore, while α determines the overall performance regime of PCE, the optimal performance for a given configuration (α, c) is critically dependent on the choice of β . If β is not appropriately tuned, the algorithm fails to satisfy the constraint and produces suboptimal solutions. Moreover, since the optimal value of β varies with c , these observations motivate the search for a functional dependence $\beta(c)$.

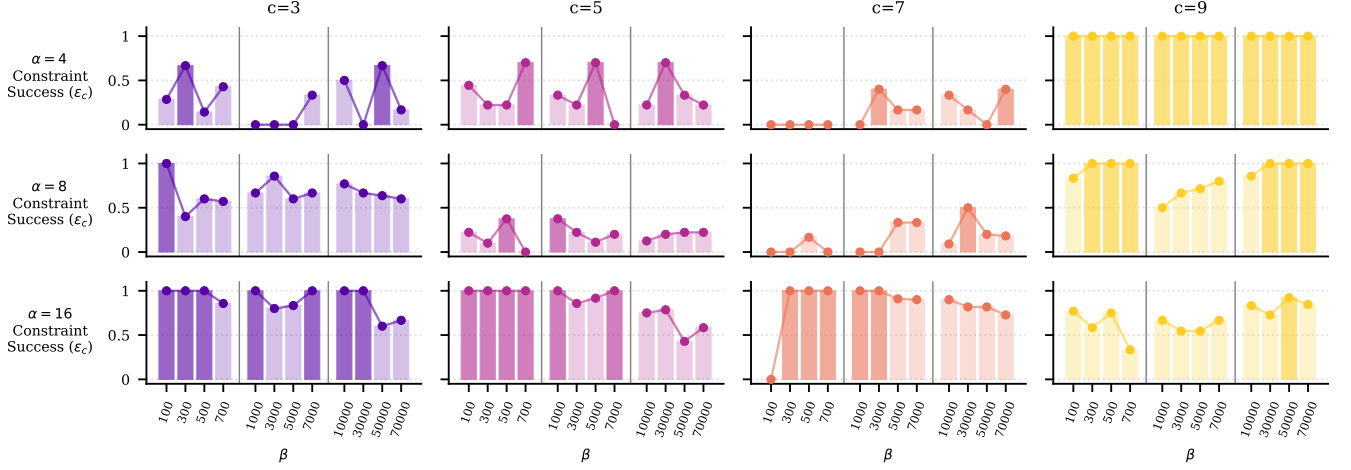


FIG. 5: Constraint success ratio ε_c for different configurations of (α, β, c) on an 18-node graph. From top to bottom, each row corresponds to $\alpha \in \{4, 8, 16\}$, while each column corresponds to $c \in \{3, 5, 7, 9\}$. At fixed α and c , ε_c is reported for three ranges of β : $\beta \sim 10^2$, $\beta \sim 10^3$, and $\beta \sim 10^4$. The highlighted bars indicate the highest ε_c achieved for each (α, c) configuration. Each bar corresponds to the mean ratio over 10 simulations.

We propose a heuristic method to estimate an effective penalty parameter $\beta(c) = \beta_c$. In general, β must be chosen in accordance with the scale of the loss function to properly balance the constraint term against the cut objective. Before introducing the heuristic used to compute β_c , we consider a simple illustrative example: a cut with $c = 2$ in a five-node graph, shown in Figure 6. From Figure 6, the cut size obtained by separating nodes 1, 2 from nodes 3, 4, 5 is strictly smaller than the sum of the cut sizes obtained by separating nodes 1 and 2 individually. This relationship can be expressed by the following inequality:

$$d_{13} + d_{14} + d_{23} + d_{24} \leq d_{13} + d_{14} + d_{23} + d_{24} + 2d_{12} \quad (16)$$

It is important to note that this expression changes when $c = 3$, since the set of negative variables becomes $z_3 = z_4 = z_5 = -1$, even though the resulting cut may remain unchanged. For this reason, we restrict the formulation to the regime $c \in [2, m/2]$. Let us introduce the quantity

$$d_i = \sum_{j=1}^n d_{ij}, \quad (17)$$

which represents the sum of all weights connected to node i . For a fixed value of c in the constraint, let $\bar{d}_i \mid i \in n_c$ denote the set of the c largest values of d_i , where n_c indexes the corresponding nodes. Then, the following expression is always true:

$$\sum_i^{n-1} \sum_{j>i}^n \frac{1}{2} d_{ij} (1 - z_i z_j) \leq \sum_i^n \frac{1}{2} d_i (1 - z_i) \leq \sum_i^{n_c} \bar{d}_i \quad (18)$$

The full mathematical deduction can be found on Appendix. This upper bound holds for any value of c and

for any graph. Based on this result, we propose to define the penalty parameter as

$$\beta(c) = \beta_c = \sum_{i \in n_c} \bar{d}_i. \quad (19)$$

We have tested this proposed value of β_c . In Figure 7 it is shown a comparison between the constraint success ratio ε_c obtained by different configurations of α, β , and the proposed value α, β_c . From the results, two main conclusions can be drawn:

1. The overall performance is predominantly determined by α . This trend is observed across the full range of β values and is also reflected in the results obtained using β_c , where ε_c increases from 0.50 at $\alpha = 4$ to 0.92 at $\alpha = 16$.
2. The proposed penalty parameter β_c yields equal or improved performance using a single value for all c , independently of α .

In conclusion, the proposed expression for $\beta(c)$ yields consistent performance and can therefore be used to study the algorithm in a systematic manner, without being affected by performance drops associated with particular choices of β for specific (α, c) configurations.

C. Regularization term

The regularization term is introduced in the original paper as a quadratic penalty centered at zero. The term is designed to keep the expectation values far from large values, which is reported to promote smoother optimization dynamics by effectively explore a broader range of

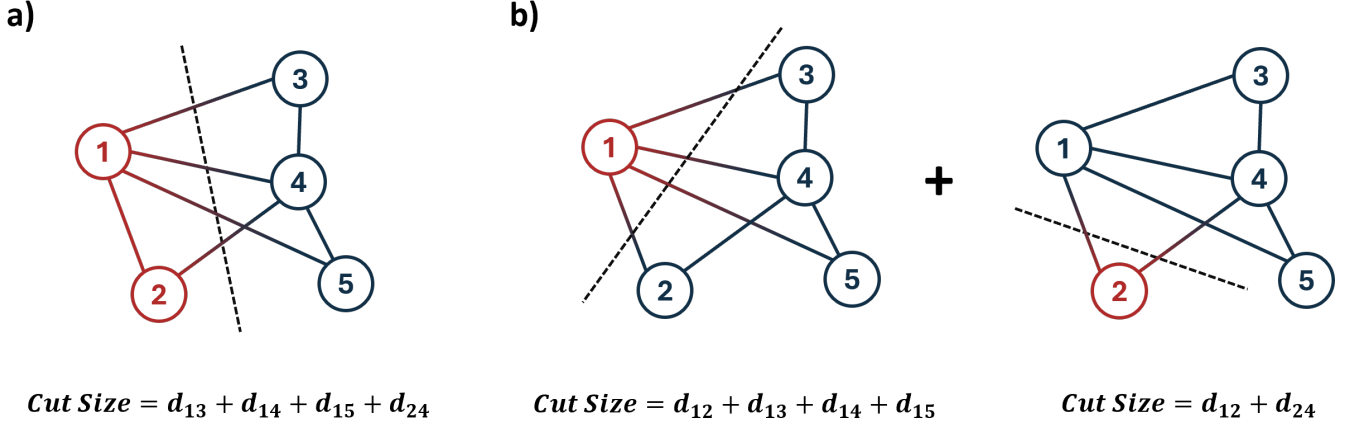


FIG. 6: Example of a cut separating two nodes ($c = 2$). The cut size of the configuration (a) is less than or equal to the sum of the cut sizes obtained by separating each node individually (b).

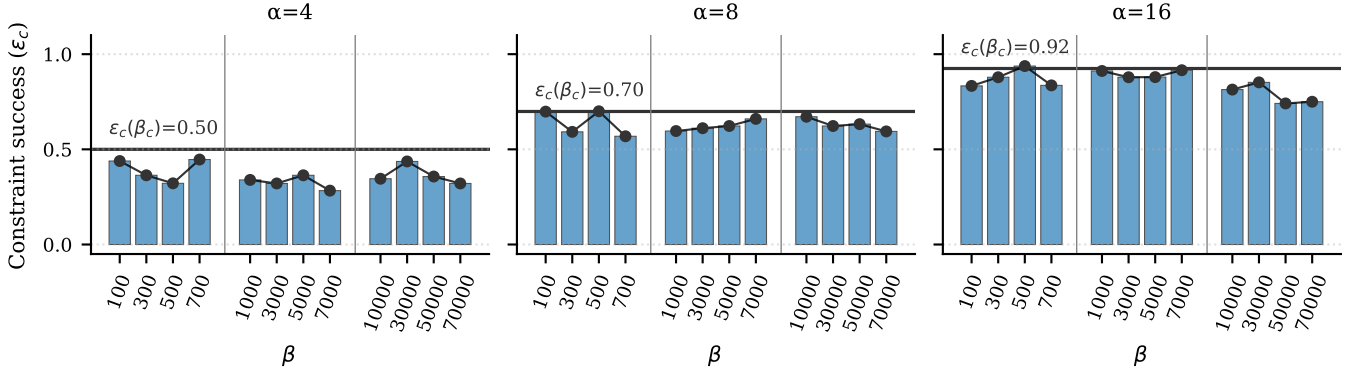


FIG. 7: Constraint success ratio ε_c for different configurations of $\{\alpha, \beta\}$ and $\{\alpha, \beta_c\}$ on the 18-node graph. Each bar represents the mean value of ε_c averaged over all executions and c values.

variable configurations. Its formulation is:

$$L^{reg} = \eta \left[\frac{1}{n} \sum_i^n \tanh(\alpha \langle \Pi_i \rangle) \right]^2 \quad (20)$$

where n denotes the number of nodes and acts as a normalization factor, and $\eta > 0$ controls the strength of the regularization. In the original PCE paper, η is analytically estimated for the Max-Cut problem; however, in more general settings it is typically chosen to be of the same order of magnitude as the loss function. In our problem, this term may interfere with constraint satisfaction, as the latter requires fully binarized variables. To assess its impact, we have performed simulations across a range of values of η for several graph instances. As it is shown in the results on Figure 8, as the strength η used in the regularization term, the constraint success ratio decreases. In parallel, the binarization of the variables also decreases, as expected.

Therefore, we conclude that the regularization term can be counterproductive for constrained problems using the PCE.

D. Pauli strings in the encoding

The main advantage of PCE lies in its ability to compress a large number of classical variables into a smaller number of qubits by encoding them into Pauli strings. This compression, however, has important hardware implications. The full set of Pauli strings $\Pi_i \in \{X^{\otimes k} \otimes \mathbb{I}^{\otimes m-k}, Y^{\otimes k} \otimes \mathbb{I}^{\otimes m-k}, Z^{\otimes k} \otimes \mathbb{I}^{\otimes m-k}\}$ does not commute. In practice, this set can be partitioned into three mutually commuting subsets, consisting of strings containing only X -, only Y -, or only Z -Pauli operators. As a consequence, each iteration requires three separate measurement settings, increasing the measurement cost by a factor of three. To mitigate this overhead, we explore alternative encodings that use only a single Pauli type, so that all Pauli strings commute and only one measurement setting is required per iteration. In general, the number of variables that can be encoded per Pauli type is:

$$n_P = \sum (\{P^{\otimes k} \otimes \mathbb{I}^{\otimes m-k}\}) = \binom{m}{k} \quad (21)$$

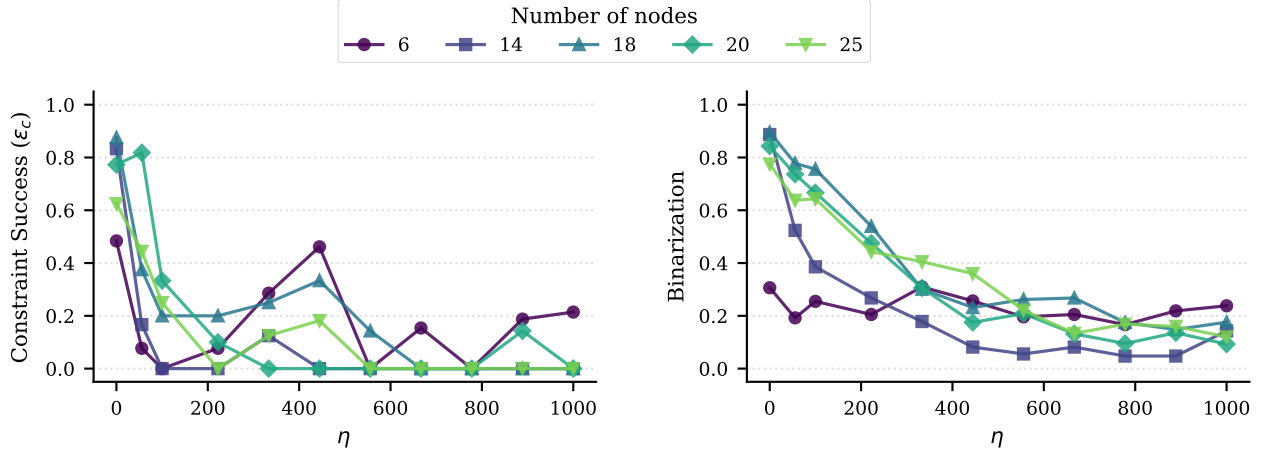


FIG. 8: Effect of the regularization strength η on constraint satisfaction and binarization. Each data point corresponds to the mean value averaged over all $c \in [2, n/2]$, 5 times executed each one.

where m is the number of qubits. Therefore, the reduction in measurement cost comes at the expense of reduced compression. For a given problem, this will affect the number of qubits needed. For example, encoding a 9-node graph with two Pauli operators per string ($k = 2$) using only Z -Paulis requires five qubits, :

$$\{\text{IIIZZ, IIZIZ, IZIIZ, ZIIIZ, IIZZI, IZIZI, ZIIZI, IZZII, ZIZII}\} \quad (22)$$

whereas using the full Pauli set $\{X, Y, Z\}$ requires only three qubits:

$$\{\text{IXX, IYY, IZZ, XIX, YIY, ZIZ, XXI, YYI, ZZI}\} \quad (23)$$

In the other hand, the effect of Pauli correlations on variable binarization is non-trivial. To investigate this effect, we compare PCE encodings using only Z -Paulis with the standard $\{X, Y, Z\}$ encoding. Since the number of parameters in the brickwork ansatz depends on the number of qubits, we use the same number of qubits in both cases to ensure a fair comparison. The results, reported in the upper half of Table III, show that using only Z -Paulis has a similar performance than using the full set of Paulis. The compression achievable with a single Pauli type can be increased by allowing Pauli strings of different orders k , i.e., by mixing values of k within the same encoding. To the best of our knowledge, mixed- k encodings have not been previously explored. If Pauli strings spanning the full range $k \in [1, m]$ are allowed, the total number of variables that can be encoded using a single Pauli type becomes

$$\sum_{k=1}^m \binom{m}{k} \quad (24)$$

For instance, with 4 qubits and only Z -Paulis, fixing $k = 2$ allows encoding $\binom{4}{2} = 6$ variables, whereas allowing all values of k increases this number to $4 + 6 + 4 + 1 = 15$.

Nodes	m	Encod.	k	ε_c	Bin.	CutSize
6	4	X,Y,Z	2	0.62	0.09	1.02
6	4	Z	2	0.73	0.08	1.01
14	6	X,Y,Z	2	0.91	0.88	1.26
14	6	Z	2	1.00	0.91	1.27
18	7	X,Y,Z	2	0.84	0.89	1.10
18	7	Z	2	0.83	0.88	1.03
6	3	X,Y,Z	2	0.66	0.11	1.01
6	3	Z	[1,4]	0.90	0.26	1.00
14	4	X,Y,Z	2	0.88	0.87	1.29
14	4	Z	[1,4]	0.80	0.79	1.36
18	5	X,Y,Z	2	0.93	0.90	1.37
18	5	Z	[1,4]	0.85	0.88	1.10

TABLE III: Results using PCE with only Z -Pauli compared with the standard X, Y, Z Pauli with different orders k . For a fair comparison, the same circuit ansatz was used for all encodings, including the number of qubits and classical parameters.

We have tested the PCE using this mixed- k encoding with only Z -Paulis. Results are shown in the lower half of Table III, where they are compared to the results using the standard X, Y, Z encoding with fixed order $k = 2$. As in the previous analysis, the same number of qubits was used in both cases. Overall, neither approach exhibits a systematic advantage across all problem sizes. Nevertheless, notable differences emerge in specific instances. For example, in the 6-node graph, the Z -only mixed- k encoding achieves a substantially higher constraint success ratio. In contrast, for the 18-node graph, the mixed- k Z -only encoding yields a lower constraint success ratio but a significantly improved cut size.

In our view, substantial work remains open regarding

the design of the encoding. Promising directions include extending mixed-order encodings to the full Pauli set X, Y, Z and scaling the present analysis to larger problem instances. We present this study as a first step intended to motivate the exploration of alternative encoding strategies within the PCE framework.

More broadly, we believe that a key open research direction lies in understanding how the choice of Pauli operators and the order of the corresponding Pauli strings influence the optimization outcome. In particular, it remains unclear whether there exists a non-trivial relationship between the structure of the optimal solution and the selected encoding, or how the encoding affects the distribution of expectation values.

V. ITERATIVE- α PCE

As discussed in the previous sections, the parameter α must be sufficiently large to induce binarization of the variables, yet not so large as to cause the optimization to stall. Here we explore an adaptive strategy to determine α dynamically rather than fixing it *a priori*. The proposed approach starts from a small initial value of α and progressively increases it, gradually driving the variables toward binarization as the optimization proceeds. In principle, this iterative procedure could naturally identify an effective value of α that balances binarization and optimization stability.

We refer to this approach as Iterative- α PCE, in which the parameter α is updated across successive stages based on feedback from the current solution. The core idea is to allow the PCE optimization to converge for an initial given value of α , and then, based on the result, compute the next value of α . The optimization is subsequently restarted from the final parameter configuration of the parametrized quantum circuit using this larger value of α . By repeating this process the variables are progressively pushed toward full binarization without abruptly entering the plateau regime of the loss landscape. In detail, the proposed heuristic for adjusting α operates as described in Algorithm 1.

We have tested the Iterative- α PCE on graphs with $n = \{6, 14, 18, 20, 25\}$ nodes. Details of both the initial value of α and the threshold value M can be found on Appendix. To disentangle the effect of the final value of α from that of the iterative procedure itself, we perform an additional control experiment. For each execution of the Iterative- α PCE, we also run a standard PCE using a fixed value α_f , where α_f corresponds to the final value reached by the iterative scheme in that same execution. This paired comparison allows us to assess whether any observed performance improvement arises solely from reaching a suitable value of α , or whether the iterative update mechanism provides an intrinsic advantage. Table IV reports, for each graph size, the percentage of runs falling into each possible combination of constraint satisfaction outcomes for the two methods.

Algorithm 1: Iterative- α PCE heuristic

Require: Initial value α_0 , threshold M

```

1:  $\alpha \leftarrow \alpha_0$ 
2: repeat
3:   Run PCE until convergence with current  $\alpha$ 
4:   Identify the set  $\mathcal{I} = \{i \mid |\tanh(\alpha\langle\Pi_i\rangle)| < M\}$ 
5:   if  $\mathcal{I} \neq \emptyset$  then
6:     Select
           
$$i^* = \arg \min_{i \in \mathcal{I}} ||\tanh(\alpha\langle\Pi_i\rangle)| - M|$$

7:   Update
           
$$\alpha \leftarrow \alpha \frac{\operatorname{arctanh}(M)}{\operatorname{arctanh}(|\tanh(\alpha\langle\Pi_{i^*}\rangle)|)}$$

8:   end if
9: until  $|\tanh(\alpha\langle\Pi_i\rangle)| \geq M \quad \forall i$ 

```

Nodes	Constraint satisfied		Runs (%)
	Iterative- α	PCE(α_f)	
6	✓	✓	53 %
	✓	×	47 %
	×	✓	0 %
	×	×	0 %
14	✓	✓	45 %
	✓	×	55 %
	×	✓	0 %
	×	×	0 %
18	✓	✓	37 %
	✓	×	63 %
	×	✓	0 %
	×	×	0 %
20	✓	✓	29 %
	✓	×	71 %
	×	✓	0 %
	×	×	0 %
25	✓	✓	22 %
	✓	×	78 %
	×	✓	0 %
	×	×	0 %

TABLE IV: Constraint satisfaction outcomes for Iterative- α PCE and PCE(α_f). Each row reports the percentage of executions corresponding to each combination of outcomes. ✓ indicates that the constraint is satisfied, while × denotes violation.

If both approaches were effectively equivalent, the fraction of runs in which only one of them satisfies the constraint would be close to zero. The results clearly deviate from this scenario. In all tested graph sizes, the percentage of runs in which the PCE satisfies the constraint while the Iterative- α PCE does not is 0%. Conversely, when the Iterative- α PCE satisfies the constraint, PCE does it only in a fraction of the runs, ranging from 53% for $n = 6$ down to 22% for $n = 25$. Notably, the proportion of exe-

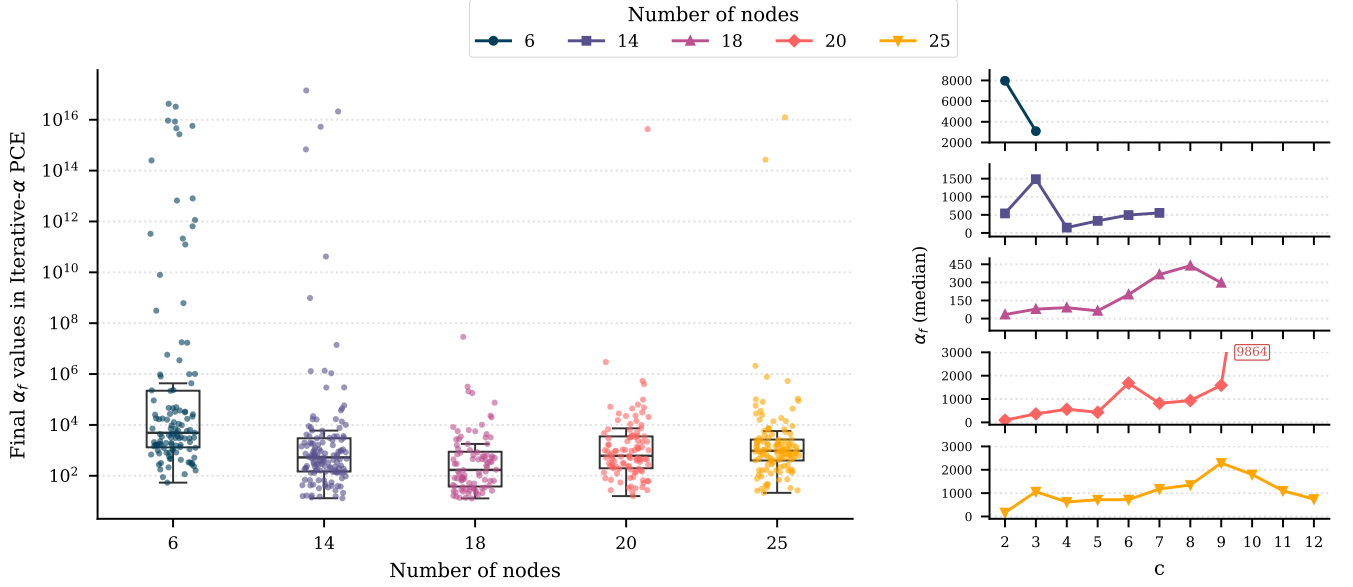


FIG. 9: Final values of α reached by the Iterative- α PCE. Left: distribution of α across executions for different graph sizes. Right: median α as a function of the constraint parameter c , with interquartile ranges.

Nodes	CutSize		
	Iterative- α	PCE (α_f)	Δ (%)
6	1.0620	1.1589	-8.36 %
14	1.3195	1.8265	-27.76 %
18	1.6523	2.5674	-35.65 %
20	1.1526	1.2543	-8.11 %
25	1.0999	1.1666	-5.72 %

TABLE V: Mean CutSize achieved (lower is better), computed over matched pairs where both methods satisfy the constraint. Δ : percentage difference of Iterative- α relative to PCE (α_f).

cutions in which the Iterative- α PCE succeeds while the PCE fails increases systematically with the graph size, reaching 78% for $n = 25$.

Regarding the CutSize, in Table V are shown the results for only those simulations in which both methods satisfy the constraint. The Iterative- α PCE consistently achieves a better solution quality, yielding cut sizes between a 5% – 30% smaller than the PCE.

Taken together, these results indicate that the observed performance gains cannot be attributed solely to the final value α_f achieved by the algorithm, but rather to the iterative evolution of α during the optimization process itself.

Figure 9 shows the distribution of final values α_f reached across all executions of the Iterative- α PCE. No clear concentration around a specific value of α_f is observed. Instead, the distribution spans several orders of

magnitude, ranging approximately from 10^2 to 10^4 , with a subset of executions reaching values as large as 10^6 and even up to 10^{16} . Consistently, when inspecting the median value of α_f as a function of the constraint strength c and graph size, no systematic trend emerges. This behavior indicates that the Iterative- α procedure does not converge toward a specific optimal value of α , but rather explores a broad range of final parameters depending on the instance and execution. Evaluating the performance of the PCE over this ensemble of final values provides a representative assessment of the behavior of standard PCE across a wide and heterogeneous range of α . Table VIII summarizes the aggregated results obtained from all executions, reporting constraint satisfaction ratio ε_c , binarization, and cut size for both the Iterative- α PCE and the corresponding PCE(α_f).

Nodes	PCE Type	ε_c	Bin.	CutSize
6	Iterative- α	1.00	1.00	1.05
6	PCE (α_f)	0.53	0.99	1.16
14	Iterative- α	1.00	1.00	1.41
14	PCE (α_f)	0.45	0.93	1.93
18	Iterative- α	1.00	1.00	1.80
18	PCE (α_f)	0.37	0.93	2.57
20	Iterative- α	1.00	1.00	1.12
20	PCE (α_f)	0.29	0.51	1.25
25	Iterative- α	1.00	1.00	1.13
25	PCE (α_f)	0.22	0.96	1.17

TABLE VI: Results using the Iterative- α PCE and the PCE with the final α_f values obtained from each run.

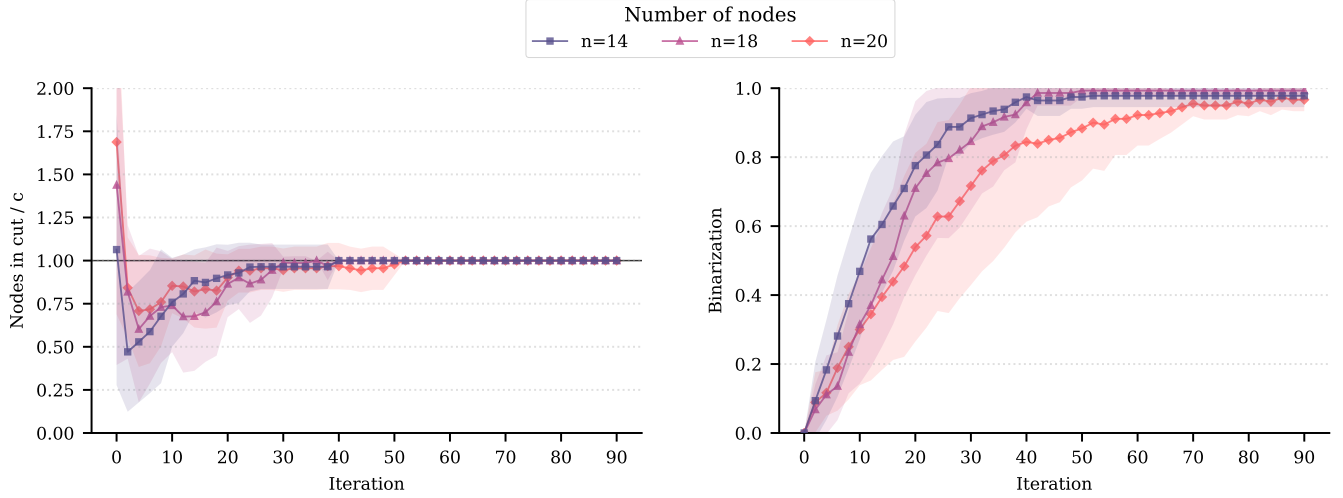


FIG. 10: Evolution of constraint satisfaction and binarization during the Iterative- α PCE. The number of nodes assigned to the cut is normalized by the target value c .

The results are unambiguous. The Iterative- α PCE achieves 100% constraint satisfaction and full binarization across all tested graph sizes. In contrast, while the PCE consistently reaches binarized solutions, constraint satisfaction is achieved only in a fraction of the executions, decreasing from 53% for small graphs down to 22% for the largest instances. This degradation with graph size is consistent with the stagnation effects observed in the standard PCE analysis. Moreover, the Iterative- α PCE also yields superior solution quality, achieving smaller cut sizes overall.

Intuitively, this behavior can be understood as follows. As α increases, a subset of variables enters the plateau region of the tanh function, while the remaining variables stay within its linear regime. As discussed previously, variables in the linear regime allow the optimizer to reduce the loss by minimizing the constraint term without truly satisfying it, while also reducing the cut size in the continuous domain. When α is increased and the optimization is restarted from the previously converged solution, the width of the plateau region expands (Figure 2). Consequently, variables that were already binarized become more deeply embedded in the plateau, making it increasingly difficult for the optimizer to move them back into the linear regime. As a result, the optimization dynamics naturally focus on the remaining variables in the linear regime that can still explore the loss landscape. As α continues to grow, more variables transition into the plateau region, and fewer variables remain able to explore continuous configurations. This progressively reduces the values available to minimize the constraint without actually satisfying it. In this way, the effective search space explored by the optimizer shrinks gradually, biasing the optimization toward configurations that fulfill the constraint. This mechanism explains why increasing α iteratively leads to a full constraint success ratio and full

binarization of the variables.

Figure 10 illustrates this process by showing the evolution of the number of nodes assigned to the cut together with the binarization metric as a function of the iteration (update of α) in the iterative procedure. The figure clearly shows how the constraint is progressively satisfied, eventually reaching the exact target value c , while the variables simultaneously become fully binarized. An example of the evolution of the variables during the iterative process is provided in the Appendix.

In conclusion, the proposed Iterative- α PCE method provides an optimization subroutine that does not require any prior estimation of the parameter α . By iteratively updating α throughout the optimization process, it consistently achieves high-quality solutions, driving the optimization toward the binary domain through full variable binarization while satisfying the constraint in 100% of the executions and systematically outperforming standard PCE.

From a hardware perspective, this iterative strategy increases the number of circuit executions required. However, the quantum circuit remains unchanged across iterations, and only the parameters of the variational ansatz are updated.

A. Large-scale simulation

Due to the outstanding results using the Iterative- α PCE, we scaled up the size of the target graphs to $n \in \{50, 150, 300\}$. Table VII summarizes the structural properties of all graph instances considered in this section. For these larger instances, we move from the Qiskit-based implementation to executing the complete Iterative- α PCE pipeline using the *Fujitsu QARP* framework, which provides an integrated and highly opti-

mized implementation of the method. For reference, Appendix E presents a runtime comparison between the QARP framework and our Qiskit-based implementation. While we do not claim that the Qiskit implementation is fully optimized for this problem, QARP is an in-house framework that enables deeper optimization and tighter workflow integration, resulting in significantly improved execution times.

Nodes	Edges	Density	Avg. Degree	Clustering	Connected
25	300	1.00	24.00	1.00	✓
50	1223	0.99	48.92	0.99	✓
150	11175	1.00	149.00	1.00	✓
300	44850	1.00	299.00	1.00	✓

TABLE VII: Key structural properties of the analyzed graphs. The final column indicates whether each graph forms a single connected component.

In this section the heuristic to change the α parameter was slightly changed from Algorithm 1 (line 7). Due to the larger number of nodes in these graphs, we observed many times the variables reached values too close to the threshold M of the heuristic. This resulted in too small changes of α that stalled the optimization. To ensure α is increased we changed the update of α between iterations to:

$$\alpha \leftarrow \alpha \frac{\text{arctanh}(M)}{|\tanh(\alpha \langle \Pi_{i^*} \rangle)|} \quad (25)$$

Further detail can be found on Appendix.

B. Results

The same analysis as for the smaller graph instances was performed. Constraint satisfaction results are reported in Table VIII, cut sizes are presented in Table IX, and aggregated metrics are summarized in Table X. The results are clear, the Iterative- α PCE consistently outperforms the standard PCE, with almost 100% of constraint satisfaction, while also yielding better cut sizes. The results for several values of the constraint parameter c are reported in the Appendix.

Furthermore, Table X reports the average number of iterations (defined as updates of the penalty parameter α) for each graph instance. Interestingly, the number of iterations seems to be independent of the graph size.

To conclude, we present in Table XI a summary of the results obtained in all the graphs studied in this paper.

VI. CONCLUSIONS AND FUTURE WORK

In this work, we conducted an in-depth study of the PCE approach for a constrained combinatorial optimization problem. Our analysis identified that the binarization of the variables plays a crucial role in obtaining valid

Nodes	Constraint satisfied		Runs (%)
	Iterative- α	PCE(α_f)	
25	✓	✓	16 %
	✓	×	84 %
	×	✓	0 %
	×	×	0 %
50	✓	✓	20 %
	✓	×	80 %
	×	✓	0 %
	×	×	0 %
150	✓	✓	8 %
	✓	×	87 %
	×	✓	0 %
	×	×	5 %
300	✓	✓	25 %
	✓	×	63 %
	×	✓	0 %
	×	×	12 %

TABLE VIII: Constraint satisfaction outcomes for Iterative- α PCE and PCE(α_f). Each row reports the percentage of executions corresponding to each combination of outcomes. ✓ indicates that the constraint is satisfied, while × denotes violation.

Nodes	CutSize		
	Iterative- α	PCE (α_f)	Δ (%)
25	1.0290	1.1690	-11.97%
50	1.0932	1.2407	-11.89%
150	1.0722	1.1075	-3.19%
300	1.0001	1.0077	-0.75%

TABLE IX: Mean CutSize achieved (lower is better), computed over matched pairs where both methods satisfy the constraint. Δ : percentage difference of Iterative- α PCE relative to PCE (α_f).

Nodes	PCE Type	ε_c	Bin.	CutSize	Iter.
25	Iterative- α	1.00	0.99	1.04	19
25	PCE (α_f)	0.16	0.99	1.17	-
50	Iterative- α	1.00	1.00	1.16	16
50	PCE (α_f)	0.20	1.00	1.24	-
150	Iterative- α	0.95	0.99	1.22	16
150	PCE (α_f)	0.08	1.00	1.11	-
300	Iterative- α	0.88	0.98	1.01	17
300	PCE (α_f)	0.25	1.00	1.01	-

TABLE X: Results using the Iterative- α PCE and the PCE with the final α_f values obtained from each run.

Nodes	Qubits	Constraint Success		Δ CutSize
		Iterative- α	PCE	Iterative- α vs PCE
6	3	100 %	53 %	-8.36 %
14	4	100 %	45 %	-27.76 %
18	4	100 %	37 %	-35.65 %
20	5	100 %	29 %	-8.11 %
25	5	100 %	16 %	-11.97 %
50	6	100 %	20 %	-11.89 %
150	8	94 %	13 %	-3.19 %
300	9	88 %	25 %	-0.75 %

TABLE XI: Summary of Iterative- α PCE results. Δ CutSize denotes the percentage difference in cut size relative to the standard PCE. Negative values indicate that Iterative- α achieves a smaller cut size than standard PCE.

solutions that satisfy the constraint term. Without careful optimization of the hyperparameters, the standard PCE struggles to enforce constraints, which often leads to infeasible solutions.

To address this limitation, we introduced the Iterative- α PCE, a modification of the baseline designed to handle constrained optimization problems more effectively. This approach achieves near-100 % constraint satisfaction while consistently producing better cut sizes compared to the standard PCE. Notably, using this iterative strategy, we were able to solve constrained graph instances with up to 300 nodes using only 9-qubits quantum circuits. We believe this represents a significant step forward in the application of PCE to large-scale constrained optimization problems, enabling the algorithm to be applied to more realistic problem instances while requiring a reduced number of qubits. This characteristic makes the approach particularly well suited to the NISQ era.

Nevertheless, this improvement comes at the cost of increased computational overhead. The iterative scheme typically requires on the order of 10–20 executions of individual PCE runs due to repeated updates of the parameter α . Importantly, this overhead appears to be largely independent of the number of nodes, which makes the algorithm promising from a scalability perspective. Future research should therefore focus on developing more effective heuristics to better understand the scalability of the method, as well as on reducing the number of required iterations while still ensuring constraint satisfaction. Another promising direction is the exploration of advanced encoding strategies, such as single-Pauli correlation schemes, which could reduce the total number of required quantum circuits by up to a factor of three. Together, these findings may further enhance the practicality of the PCE algorithm for constrained optimization on near-term quantum hardware.

-
- [1] A. Abbas, A. Ambainis, B. Augustino, A. Bäertschi, H. Buhrman, C. Coffrin, G. Cortiana, V. Dunjko, D. J. Egger, B. G. Elmegreen, and others, Challenges and opportunities in quantum optimization, *Nature Reviews Physics*, 1 (2024), publisher: Nature Publishing Group.
 - [2] M. Cerezo, A. Arrasmith, R. Babbush, S. C. Benjamin, S. Endo, K. Fujii, J. R. McClean, K. Mitarai, X. Yuan, L. Cincio, and others, Variational quantum algorithms, *Nature Reviews Physics* **3**, 625 (2021), publisher: Nature Publishing Group UK London.
 - [3] E. Farhi, J. Goldstone, and S. Gutmann, A quantum approximate optimization algorithm, *arXiv preprint arXiv:1411.4028* (2014).
 - [4] A. Peruzzo, J. McClean, P. Shadbolt, M.-H. Yung, X.-Q. Zhou, P. J. Love, A. Aspuru-Guzik, and J. L. O’Brien, A variational eigenvalue solver on a photonic quantum processor, *Nature communications* **5**, 4213 (2014), publisher: Nature Publishing Group UK London.
 - [5] D. d. Falco and D. Tamascelli, An introduction to quantum annealing, *RAIRO - Theoretical Informatics and Applications* **45**, 99 (2011).
 - [6] S. Yarkoni, E. Raponi, T. Bäck, and S. Schmitt, Quantum annealing for industry applications: introduction and review, *Reports on Progress in Physics* **85**, 104001 (2022), publisher: IOP Publishing.
 - [7] M. Sciorilli, L. Borges, T. L. Patti, D. García-Martín, G. Camilo, A. Anandkumar, and L. Aolita, Towards large-scale quantum optimization solvers with few qubits, *Nature Communications* **16**, 476 (2025), publisher: Nature Publishing Group UK London.
 - [8] V. P. Soloviev and M. Krompiec, Large-scale portfolio optimization using Pauli Correlation Encoding, *arXiv preprint arXiv:2511.21305* (2025).
 - [9] Pauli Correlation Encoding to reduce Maxcut requirements (), publication Title: IBM Quantum Documentation.
 - [10] Variational quantum optimization with multibasis encodings | *Phys. Rev. Research* ().

- [11] J. Puerto and J. L. Sainz-Pardo, Budget-constrained cut problems (2023), arXiv:2308.07063 [math].
- [12] R. Engelberg, J. Könemann, S. Leonardi, and J. S. Naor, Cut problems in graphs with a budget constraint, *Journal of Discrete Algorithms 2004 Symposium on String Processing and Information Retrieval*, **5**, 262 (2007).
- [13] C. Chekuri and A. Kumar, Maximum Coverage Problem with Group Budget Constraints and Applications, in *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques*, edited by K. Jansen, S. Khanna, J. D. P. Rolim, and D. Ron (Springer, Berlin, Heidelberg, 2004) pp. 72–83.
- [14] R. Takei, W. Chen, Z. Clawson, S. Kirov, and A. Vladimirovsky, Optimal Control with Budget Constraints and Resets, *SIAM Journal on Control and Optimization* **53**, 712 (2015), publisher: Society for Industrial and Applied Mathematics.
- [15] W. Shi and B. Hong, Resource Allocation with a Budget Constraint for Computing Independent Tasks in the Cloud, in *2010 IEEE Second International Conference on Cloud Computing Technology and Science* (2010) pp. 327–334.
- [16] K. Rose, E. Gurewitz, and G. Fox, Constrained clustering as an optimization method, *IEEE Transactions on Pattern Analysis and Machine Intelligence* **15**, 785 (1993).
- [17] P. J. Van Laarhoven and E. H. Aarts, Simulated annealing, in *Simulated annealing: Theory and applications* (Springer, 1987) pp. 7–15.
- [18] J. A. Nelder and R. Mead, A Simplex Method for Function Minimization, *The Computer Journal* **7**, 308 (1965).
- [19] Exact and sequential penalty weights in quadratic unconstrained binary optimisation with a digital annealer | Proceedings of the Genetic and Evolutionary Computation Conference Companion ().
- [20] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, and others, Quantum computing with Qiskit, arXiv preprint arXiv:2405.08810 (2024).

Appendix A: β_c calculus

We introduce here the full mathematical development of the $\beta(c)$ expression. Let $x_i \in \{-1, 1\}$. For convenience, we first rewrite the CutSize term as:

$$\begin{aligned} \sum_{i=1}^{n-1} \sum_{j>i}^n \frac{1}{2} d_{ij} (1 - x_i x_j) &= \sum_{i=1}^{n-1} \sum_{j>i}^n \frac{1}{2} d_{ij} (1 - x_i x_j + x_i - x_i) = \\ &= \sum_{i=1}^{n-1} \sum_{j>i}^n \frac{1}{2} d_{ij} (1 - x_i) + \sum_{i=1}^{n-1} \sum_{j>i}^n \frac{1}{2} d_{ij} x_i (1 - x_j) \end{aligned} \quad (\text{A1})$$

given $d_{ij} = d_{ji}$, then:

$$\sum_i \sum_{j>i} \frac{1}{2} d_{ij} (1 - x_i x_j) = \sum_i \sum_{j \neq i} \frac{1}{4} d_{ij} (1 - x_i) + \sum_i \sum_{j \neq i} \frac{1}{4} d_{ij} x_i (1 - x_j) \quad (\text{A2})$$

now, the second term:

$$\begin{aligned} \sum_i \sum_{j \neq i} \frac{1}{4} d_{ij} (1 - x_i) (1 - x_j) &= \sum_i \sum_{j \neq i} \frac{1}{4} d_{ij} (1 - x_i - x_j + x_i x_j) = \\ &= \sum_i \sum_{j \neq i} \frac{1}{4} d_{ij} (1 - x_i) - \sum_i \sum_{j \neq i} \frac{1}{4} d_{ij} x_j (1 - x_i) \end{aligned} \quad (\text{A3})$$

given that $d_{ij} = d_{ji}$,

$$\sum_i \sum_{j \neq i} \frac{1}{4} d_{ij} (1 - x_i) (1 - x_j) = \sum_i \sum_{j \neq i} \frac{1}{4} d_{ij} (1 - x_i) - \sum_i \sum_{j \neq i} \frac{1}{4} d_{ij} x_i (1 - x_j) \quad (\text{A4})$$

now, introducing this in Eq. A3:

$$\begin{aligned} \sum_i \sum_{j>i} \frac{1}{2} d_{ij} (1 - x_i x_j) &\leq \\ &\leq \sum_i \sum_{j \neq i} \frac{1}{4} d_{ij} (1 - x_i) + \sum_i \sum_{j \neq i} \frac{1}{4} d_{ij} x_i (1 - x_j) + \sum_i \sum_{j \neq i} \frac{1}{4} d_{ij} (1 - x_i) - \sum_i \sum_{j \neq i} \frac{1}{4} d_{ij} x_i (1 - x_j) \end{aligned} \quad (\text{A5})$$

finally,

$$\sum_i \sum_{j>i} \frac{1}{2} d_{ij} (1 - x_i x_j) \leq \sum_i \sum_{j \neq i} \frac{1}{2} d_{ij} (1 - x_i) \quad (\text{A6})$$

The second term corresponds exactly to the sum of the weights of all edges incident to node i for all variables satisfying $x_i = -1$, that is, for the c nodes separated by the cut. When applied to the illustrative example discussed above, this expression recovers Eq. (16).

We now introduce the quantity

$$d_i = \sum_{j=1}^n d_{ij}, \quad (\text{A7})$$

which represents the sum of all weights connected to node i . For a fixed value of c in the constraint, let $\bar{d}_i | i \in n_c$ denote the set of the c largest values of d_i , where n_c indexes the corresponding nodes. It then follows that:

$$\sum_i \sum_{j>i} \frac{1}{2} d_{ij} (1 - x_i x_j) \leq \sum_i \frac{1}{2} d_i (1 - x_i) \leq \sum_i^{n_c} \bar{d}_i \quad (\text{A8})$$

Appendix B: Iterative- α PCE execution

In Figure 11 we show the evolution of the variables throughout the full execution of the Iterative- α PCE. As α increases, most variables rapidly approach their binarized values and remain in the plateau once reached. In contrast, a small subset of variables continues to explore the loss landscape more deeply, exhibiting larger fluctuations until binarization is ultimately achieved.

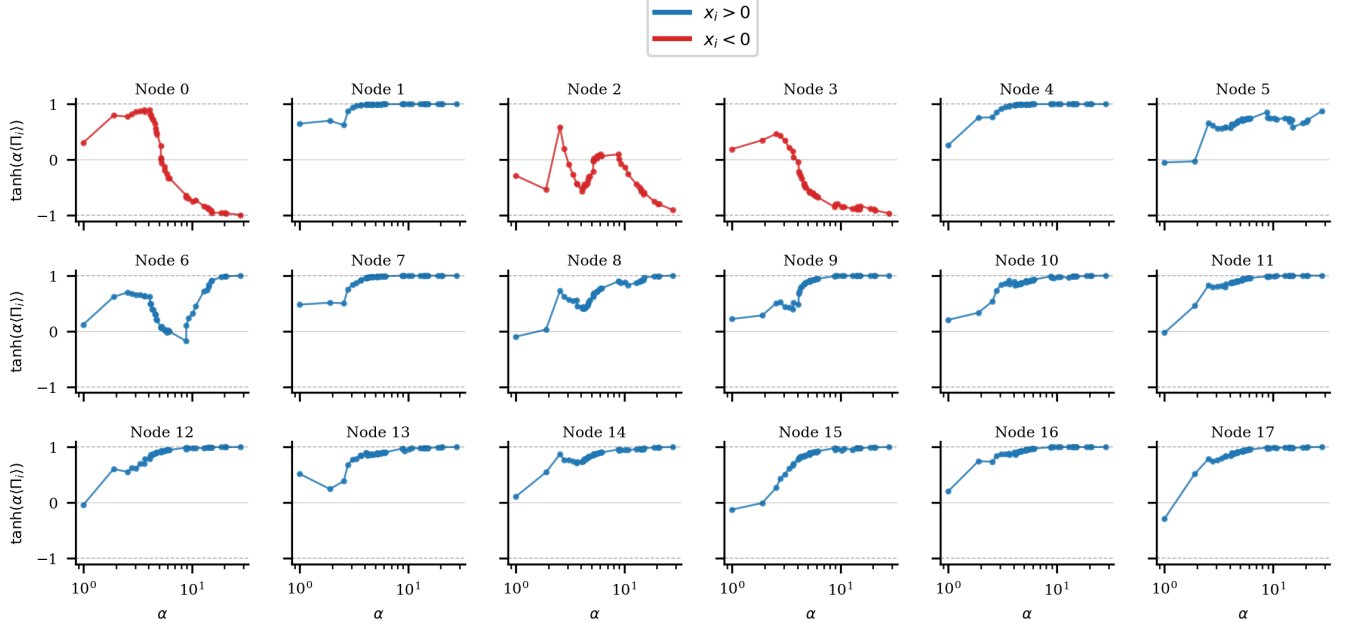


FIG. 11: Evolution of the encoded variables during a single execution of the Iterative- α PCE. Each curve represents $\tanh(\alpha\langle\Pi_i\rangle)$ as a function of α .

Appendix C: Iterative- α PCE parameters

In Table XII we report the specific parameter used in all the Iterative- α executions. For the 150- and 300-node graph instances, we increased the constraint threshold. Due to the significantly larger number of possible variable combinations, we observed that some configurations with values as high as 0.90 were still able to violate the constraint. In addition, the initial value of the penalty parameter α was reduced to 1, as we found that using $\alpha = 3$ caused the PCE to struggle to converge and to effectively minimize the constraint value itself, even when the constraint was not yet satisfied.

Nodes	Qubits	Order (k)	Threshold (M)	Initial α
6	3	2	0.90	3
14	4	2	0.90	3
18	4	2	0.90	3
20	5	2	0.90	3
25	5	2	0.90	3
50	6	3	0.90	3
150	8	4	0.95	1
300	9	4	0.95	1

TABLE XII: Parameters used on the Iterative- α PCE executions

Appendix D: Large-scale analysis

Figure 12 shows a detailed performance comparison between the single PCE execution and the Iterative- α PCE method for $n \in \{25, 50, 150, 300\}$ nodes. Each subplot reports the degree of constraint satisfaction for both approaches, for different values of the constraint parameter c , with single execution PCE shown in green and Iterative- α PCE in blue. Results are consistent regardless the level of constraint imposed in each problem instance. Iterative PCE consistently converges to higher-quality solutions, reaching full constraint satisfaction in most instances, while the single execution variant increasingly struggles as the problem size grows. These results underscore the effectiveness of the Iterative- α PCE strategy for constrained optimization problems, particularly when scalability and solution reliability are essential.

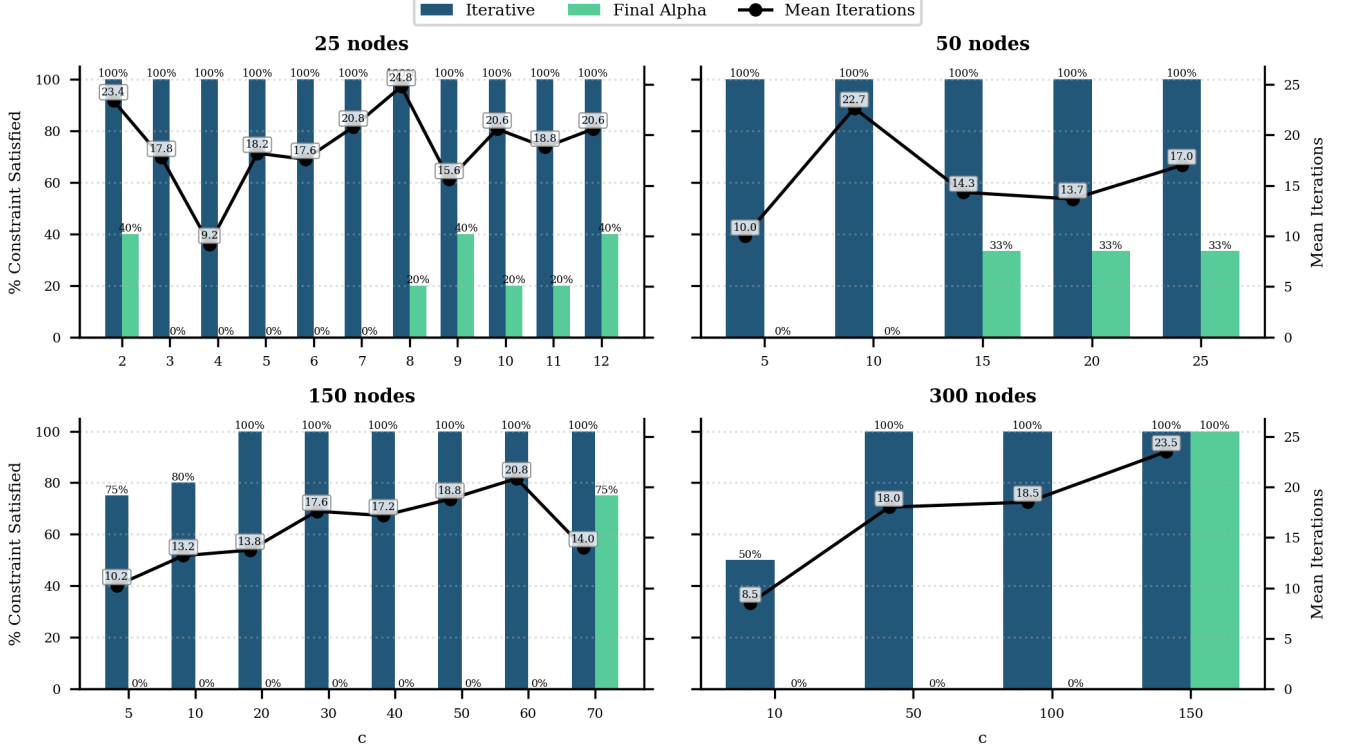


FIG. 12: Performance comparison between iterative- α method and the single execution of the PCE approach with the final parameters found by the iterative- α method for $n \in \{18, 50, 150, 300\}$

Appendix E: Runtime analysis

Table XIII compares PCE runtimes between the QARP framework (using Qulacs backend) and Qiskit [20]. Across all tested problem sizes, QARP consistently achieves lower execution times, while Qiskit exhibits a rapid increase as the number of nodes and qubits grows. This performance gap becomes more pronounced for larger instances.

Nodes	Qubits	QARP (s)	Qiskit (s)
6	3	5.52 ± 0.42	6.11 ± 4.47
18	4	11.24 ± 0.81	27.08 ± 12.12
25	5	17.44 ± 1.26	44.16 ± 18.52
50	7	39.29 ± 3.98	199.44 ± 113.38

TABLE XIII: Runtime comparison between QARP and Qiskit.